

Bayesian Machine Learning in Quantitative Finance

Chapter 2: Background

Based on Mongwe, Mbuva & Marwala

Springer, 2025

Chapter Outline

- 1 2.1 Introduction
- 2 2.1.2 Bayesian ML Models
- 3 2.1.3 Training Bayesian ML Models
- 4 2.2 Performance Metrics
- 5 2.3 Conclusion

Two Ways to Do Statistics: Frequentist vs. Bayesian I

Intuition first. Suppose you want to know how likely a coin is to land heads. There are two fundamentally different philosophies about what “probability” even means here, and they lead to two different styles of inference.

Frequentist inference

Probability is a *long-run frequency*: “the probability of heads is 0.6” means “if I flipped this coin infinitely many times, 60% of flips would be heads.” Model parameters are treated as *fixed but unknown constants*. We estimate them with a single “best guess” (a point estimate) computed from the observed data.

Bayesian inference

Probability also describes a *degree of belief*. Model parameters are treated as *random variables* with their own distribution. We start with a prior belief, update it with observed data via Bayes’ theorem, and end up with a full *posterior distribution* over the parameter – not just one number.

Two Ways to Do Statistics: Frequentist vs. Bayesian II

A little history. Bayesian inference is actually *older* than the frequentist paradigm, but it was neglected for a long time. The primary reason: Bayesian statistics could only be applied in a limited number of situations where a conjugate prior was known in closed form; when no closed-form posterior was available, the method stalled.

The resurgence of Bayesian statistics in recent decades is attributed to:

- the rapid growth of **computational power** and **data volume**, and
- the development of novel mathematical algorithms – most notably **Markov Chain Monte Carlo (MCMC)** techniques, developed in the 1950s, including the Hastings (1970) algorithm.

MCMC made it possible to build random sampling algorithms for a probability distribution and compute Bayesian hierarchical models even when no closed-form posterior exists. A few years later, statisticians began routinely using MCMC, marking the start of the contemporary Bayesian statistics era.

Two Ways to Do Statistics: Frequentist vs. Bayesian III

Criticisms on both sides. Neither paradigm is free of controversy:

- Frequentist hypothesis testing is frequently criticized for its use of p -values, which are partially based on data that were *never actually observed* (hypothetical repeated sampling).
- Bayesian methods do not use null/alternative hypotheses at all, so that particular criticism does not apply to them. Instead, the main criticism of Bayesian methods is the **subjectivity of the chosen prior distribution**: there is no single, well-defined, recommended way to pick a prior, so different studies may choose different priors for the same experiment and reach different conclusions.
- In defence of Bayesian methods: as long as the prior is appropriately specified, validity is preserved regardless of the experimental design, and Bayesian techniques merely ascertain an event's likelihood. Trying several different priors also lets us measure how *sensitive* our conclusions are to prior assumptions.

Two Ways to Do Statistics: Frequentist vs. Bayesian IV

Why this book uses the Bayesian paradigm. The Bayesian framework lets us naturally answer questions that matter a great deal in quantitative finance:

- How can we explain the prediction or output of a model?
- What is the distribution of the model's parameters (i.e. how *uncertain* are we about them)?
- How do we select between different models in a statistically principled manner?
- Which inputs are most relevant to the task at hand?

The Bayesian framework is founded on **Bayes' theorem**, which lets us update prior beliefs through new observations, encapsulated through the assumed underlying model. The rest of this chapter builds up the vocabulary needed for every later chapter in this book.

Notation You'll Need: A Glossary for the Rest of This Book I

This chapter introduces several terms that will be used, without re-explanation, in every subsequent chapter. Read this glossary carefully.

Bayesian Machine Learning (BML)

Ordinary (“frequentist”) machine learning fits a model and returns *one best-fit answer* for each parameter and each prediction. **Bayesian machine learning blends computational models with Bayesian inference:** it fits the model *and* returns a full *distribution* describing how uncertain we are about every parameter and every prediction. Analogy: instead of a weather forecaster saying “it will be 23°C tomorrow” (a point estimate), a Bayesian forecaster says “there’s a range of likely temperatures, centred around 23°C, and here is exactly how spread out that range is.”

Notation You'll Need: A Glossary for the Rest of This Book II

Prior, Likelihood, Posterior (recap)

- **Prior** $\mathbb{P}(\mathbf{w})$: what we believe about the parameters $\mathbf{w}$ *before* seeing any data.
- **Likelihood** $\mathbb{P}(D \mid \mathbf{w})$: how probable the observed data D is, for a given setting of $\mathbf{w}$.
- **Posterior** $\mathbb{P}(\mathbf{w} \mid D)$: what we believe about $\mathbf{w}$ *after* seeing the data D – obtained by combining the prior and the likelihood via Bayes' theorem.

Analogy: a detective (prior suspicion) updates their beliefs about “who did it” as new evidence (likelihood of the evidence given each suspect) comes in, arriving at a final, evidence-weighted verdict (posterior).

Notation You'll Need: A Glossary for the Rest of This Book III

Markov Chain Monte Carlo (MCMC) – high-level idea

Very often, the posterior distribution cannot be written down in closed form. **MCMC is a way to draw samples from a complicated posterior distribution you can't write down in closed form, by running a cleverly designed random walk** whose long-run behaviour matches that posterior. Instead of computing the posterior analytically, we generate a long sequence of parameter values (a “chain”) such that, after enough steps, the values visited are distributed *as if* drawn directly from the true posterior. We will make this precise (the Metropolis-Hastings algorithm) later in this chapter.

Notation You'll Need: A Glossary for the Rest of This Book IV

Gaussian Process (GP) – previewed here, used in later chapters

A Gaussian process is a “**distribution over functions**”: a flexible, non-parametric Bayesian model. Instead of assuming the relationship between inputs and outputs has a fixed parametric form (like a straight line), a GP places a prior directly over the space of possible functions, and Bayes' theorem updates this prior into a posterior distribution over functions that are consistent with the observed data points. Analogy: rather than fitting one curve, imagine an infinite bundle of candidate curves, each with a plausibility weight; the GP keeps track of that whole weighted bundle, and shrinks the bundle around the data points that have been observed.

Notation You'll Need: A Glossary for the Rest of This Book V

Normalizing Flow – previewed here, used in later chapters

A normalizing flow is **a way to build complex, flexible probability distributions by transforming a simple one, like a Gaussian, through a sequence of invertible functions**. Because each transformation is invertible (and its Jacobian determinant is easy to compute), we can start with an easy distribution (e.g. a standard Gaussian), push samples through a chain of learned transformations, and end up with samples from a much richer, more realistic distribution – while still being able to evaluate exact densities. Analogy: think of modelling clay (the simple Gaussian) that gets pushed, stretched, and twisted by a sequence of reshaping tools (the invertible transforms) into an intricate final sculpture (the complex target distribution).

Notation You'll Need: A Glossary for the Rest of This Book VI

Bayesian Neural Network (BNN) – previewed here, used in later chapters

An ordinary neural network learns one fixed number for every weight. A **Bayesian neural network is a neural network whose weights are themselves treated as uncertain, with a distribution over them, not fixed numbers**. Training a BNN means inferring a posterior distribution over all of the network's weights given the data, instead of finding a single “best” weight vector. Predictions are then made by averaging over many plausible networks (one per weight sample), which naturally produces a measure of predictive uncertainty alongside the prediction itself.

Python: Frequentist Point Estimate vs. Bayesian Posterior

Goal: on 5 coin flips (H, T, H, H, T), contrast a frequentist point estimate with a full Bayesian posterior.

```
1 import numpy as np
2 from scipy.stats import beta
3
4 data = [1, 0, 1, 1, 0]          # H,T,H,H,T encoded as 1/0
5 n = len(data)
6 n_heads = sum(data)             # 3
7 n_tails = n - n_heads          # 2
8
9 # --- Frequentist: Maximum Likelihood Estimate (MLE) ---
10 theta_mle = n_heads / n
11 print(f"Frequentist point estimate: theta_MLE = {theta_mle:.4f}")
12
13 # --- Bayesian: Beta(1,1) uniform prior -> Beta posterior ---
14 a_prior, b_prior = 1.0, 1.0
15 a_post, b_post = a_prior + n_heads, b_prior + n_tails    # Beta(4,3)
16 post_mean = a_post / (a_post + b_post)
17 ci_low, ci_high = beta.ppf([0.025, 0.975], a_post, b_post)
18
19 print(f"Bayesian posterior: Beta({a_post:.0f},{b_post:.0f})")
20 print(f"    mean = {post_mean:.4f}, 95% CI = [{ci_low:.4f}, {ci_high:.4f}])")
```

Output: theta_MLE = 0.6000; Bayesian posterior Beta(4,3), mean = 0.5714, 95% CI = [0.2228, 0.8819]. Same data, one number vs. a whole distribution with a quantified uncertainty.

2.1.1 Bayesian Machine Learning I

Definition. To produce predictions *and* their associated uncertainties, Bayesian machine learning blends computational models with the ideas of Bayesian inference. Its foundation is the Bayesian framework, which enables modelling uncertainty and updating beliefs in light of observed data and past knowledge. Bayesian machine learning combines probability distributions over model parameters *and* predictions, offering a more thorough account of uncertainty than classic machine learning techniques, which only produce point estimates.

Benefits

- Flexible modelling.
- The capacity to incorporate prior information.
- Systematic, principled treatment of uncertainty.

2.1.1 Bayesian Machine Learning II

Difficulties

- Scalability to large datasets and high-dimensional models.
- Computing complexity – Bayesian posteriors are often expensive to obtain.

Where the field is headed. Future research in Bayesian machine learning aims to:

- develop scalable algorithms,
- increase computational efficiency,
- bridge the gap between Bayesian approaches and deep learning, and
- address interpretability difficulties.

In general, Bayesian machine learning offers a practical paradigm for describing uncertainty and producing well-informed predictions in machine learning tasks. The next table summarizes how it compares to the classic (frequentist) paradigm across five practical dimensions.

Table 2.1: Frequentist vs. Bayesian Machine Learning

Metric	Frequentist	Bayesian
Uncertainty Quantification	No natural notion of uncertainty – predictions are point estimates	Captures uncertainty in parameters and predictions through probability distributions
Prior Beliefs	No notion of a prior; cannot naturally incorporate prior beliefs	A prior distribution captures beliefs about parameters, updated via Bayes' theorem
Regularization & Over-fitting	No natural regularization; the user must select a penalty by hand	Prior distributions over parameters naturally regularize, reducing over-fitting
Model Selection	Usually via K -fold cross-validation or leave-one-out; no agreed state-of-the-art metric	The Bayesian evidence (the posterior's normalizing constant) gives a principled way to compare models
Scalability	Scales more easily to huge dimensions and datasets	Often computationally demanding; can be difficult to scale to large datasets

2.1.2 Bayesian Machine Learning Models: An Overview I

Bayesian machine learning contains an assortment of methodologies that apply Bayesian principles to draw inferences and make predictions. This book uses the following model classes.

Bayesian Logistic Regression

Adds prior distributions over the parameters of classical logistic regression. In light of the observed data, it offers a *posterior distribution* across the parameters and quantifies uncertainty in the parameter estimates (rather than a single fitted coefficient vector).

Bayesian Mixture Models

Probabilistic models that assume the data is generated from a combination of underlying distributions. They handle uncertainty in the model parameters and enable density estimation and clustering by imposing priors on the mixture proportions and on the parameters of each component distribution.

2.1.2 Bayesian Machine Learning Models: An Overview II

Gaussian Processes (GPs)

Non-parametric models that are flexible and describe a *prior distribution over functions* (see the glossary earlier in this chapter). GPs can identify intricate patterns in the data and offer estimates of prediction uncertainty. Common uses: regression, classification, and time series analysis.

Bayesian Neural Networks (BNNs)

Add prior distributions over the network weights, expanding on the architecture of classical (frequentist) neural networks. BNNs can estimate posterior distributions over the weights using methods like variational inference or MCMC sampling, allowing for **Bayesian model averaging** and uncertainty quantification.

2.1.2 Bayesian Machine Learning Models: An Overview III

Bayesian Formulation of Economic Models

Sometimes a traditional (frequentist) model from quantitative finance – for example, the Heston stochastic volatility model – is instead framed within the Bayesian framework, so that its parameters get a full posterior distribution rather than a single calibrated value. These models are explored in detail in the relevant application chapters of this book.

Looking ahead. Two further tools appear repeatedly in later chapters of this book even though they are not part of this introductory list: **normalizing flows** (for building flexible, exact-density distributions, used e.g. for option-price and audit-outcome modelling) and, more generally, deep **Bayesian neural networks** for claims/insurance-type problems. Their plain-English definitions are in the glossary earlier in this chapter – the relevant chapters develop the full technical machinery.

Python: A Gaussian Process is a “Distribution over Functions”

Goal: draw one random function from a GP prior with a radial-basis-function (RBF) kernel, to make the “distribution over functions” idea concrete.

```
1 import numpy as np
2
3 def rbf_kernel(x1, x2, length_scale=1.0, variance=1.0):
4     sq_dist = (x1[:, None] - x2[None, :]) ** 2
5     return variance * np.exp(-0.5 * sq_dist / length_scale**2)
6
7 x = np.linspace(-3, 3, 50)
8 K = rbf_kernel(x, x) + 1e-6 * np.eye(len(x))    # jitter for stability
9 L = np.linalg.cholesky(K)                       # K = L L^T
10
11 rng = np.random.default_rng(0)
12 z = rng.standard_normal(len(x))                 # standard normal "noise"
13 f_sample = L @ z                                # one random FUNCTION, f(x) for all x
14
15 print("A single draw from GP(0, K) at 5 grid points:")
16 print(np.round(f_sample[:10], 3))
```

Every re-run of `rng.standard_normal` produces an *entire new curve* $f(x)$ consistent with the kernel's smoothness assumptions – literally sampling a function, not a number.

2.1.3 Training Bayesian Machine Learning Models I

The Bayesian framework, and hence Bayesian machine learning, is founded on Bayes' theorem, which states that for a given model M with parameters $\mathbf{w}$ and data D :

Bayes' Theorem for Model Parameters

$$\underbrace{\mathbb{P}(\mathbf{w} \mid D, M)}_{\text{posterior}} = \frac{\overbrace{\mathbb{P}(D \mid \mathbf{w}, M)}^{\text{likelihood}} \overbrace{\mathbb{P}(\mathbf{w} \mid M)}^{\text{prior}}}{\underbrace{\mathbb{P}(D \mid M)}_{\text{evidence}}} \quad (2.1)$$

- $\mathbb{P}(\mathbf{w} \mid D, M)$ – the **posterior**: what we believe about the parameters after seeing the data.
- $\mathbb{P}(D \mid \mathbf{w}, M)$ – the **likelihood**: how probable the data D is, for a given $\mathbf{w}$.
- $\mathbb{P}(\mathbf{w} \mid M)$ – the **prior**: what we believe about $\mathbf{w}$ before seeing D .
- $\mathbb{P}(D \mid M)$ – the **Bayesian evidence** (or marginal likelihood) for model M ; a normalizing constant that can also be used for model selection.

2.1.3 Training Bayesian Machine Learning Models II

A Bayesian model is fully specified once the prior $\mathbb{P}(\mathbf{w} \mid M)$ and the likelihood $\mathbb{P}(D \mid \mathbf{w}, M)$ have been chosen. The prior is typically a Gaussian distribution (sometimes with a different standard deviation per parameter). The likelihood is typically the Bayesian analogue of whatever loss function would have been used in frequentist inference – e.g. for a regression problem, the likelihood corresponds to the least-squares error.

2.1.3 Training Bayesian Machine Learning Models III

Why we need numerical methods. The aim of training a Bayesian machine learning model is to infer the posterior $\mathbb{P}(\mathbf{w} \mid D, M)$. This posterior is typically *not* available in closed form, because the Bayesian evidence $\mathbb{P}(D \mid M)$ is itself usually intractable (it requires integrating the numerator of Eq. (2.1) over all possible $\mathbf{w}$). Numerical techniques are therefore required. The two common families of approaches, both of which amount to determining the posterior distribution within the Bayesian framework, are:

- **Variational Inference (VI)** – turns the sampling problem into an optimization problem.
- **Markov Chain Monte Carlo (MCMC)** – draws samples directly via a guided random walk.

MCMC algorithms are generally preferable in that they converge to the *correct* target distribution if the sample size is sufficiently large, whereas MCMC's main disadvantage is slow execution compared to variational methods. Typically, neither family simultaneously estimates the Bayesian evidence *and* the posterior during inference – the evidence is usually only computed afterwards, as a post-processing step (although some methods, such as nested sampling, aim to estimate the evidence directly and produce the posterior as a by-product).

A remark on scope: this chapter (and this book) focuses on VI and MCMC as the two training paradigms. A third approximate technique you may encounter elsewhere in the wider Bayesian

2.1.3 Training Bayesian Machine Learning Models IV

machine learning literature is the **Laplace approximation**, which fits a Gaussian centred at the posterior mode using the local curvature there; it is fast but is not developed further in this chapter.

Worked Example: MLE vs. Bayesian Posterior, by Hand I

Setup. 5 coin flips: H, T, H, H, T $\Rightarrow$ 3 heads, 2 tails ($n = 5$). Model: each flip is Bernoulli(θ) with $\theta = \mathbb{P}(\text{heads})$.

Step 1 – Frequentist point estimate (MLE)

The likelihood of the data is $\mathbb{P}(D \mid \theta) = \theta^3(1 - \theta)^2$. Maximizing over θ (set derivative of the log-likelihood to zero) gives

$$\hat{\theta}_{MLE} = \frac{n_{\text{heads}}}{n} = \frac{3}{5} = 0.6.$$

This single number is the frequentist's entire answer – no uncertainty is attached to it.

Worked Example: MLE vs. Bayesian Posterior, by Hand II

Step 2 – Bayesian posterior (Beta-Bernoulli conjugacy)

Choose a uniform prior $\theta \sim \text{Beta}(\alpha_0=1, \beta_0=1)$ (i.e. we believe every value of $\theta \in [0, 1]$ is equally likely beforehand). The Beta distribution is *conjugate* to the Bernoulli likelihood, so the posterior is again Beta, with parameters updated by simply adding the observed counts:

$$\alpha_{\text{post}} = \alpha_0 + n_{\text{heads}} = 1 + 3 = 4, \quad \beta_{\text{post}} = \beta_0 + n_{\text{tails}} = 1 + 2 = 3.$$

So $\theta \mid D \sim \text{Beta}(4, 3)$: a whole distribution, not a single number.

Worked Example: MLE vs. Bayesian Posterior, by Hand III

Step 3 – Reading off numbers from the posterior

$$\mathbb{E}[\theta \mid D] = \frac{\alpha_{\text{post}}}{\alpha_{\text{post}} + \beta_{\text{post}}} = \frac{4}{7} \approx 0.5714, \quad \text{mode} = \frac{\alpha_{\text{post}} - 1}{\alpha_{\text{post}} + \beta_{\text{post}} - 2} = \frac{3}{5} = 0.6.$$

$$\text{Var}(\theta \mid D) = \frac{\alpha_{\text{post}} \beta_{\text{post}}}{(\alpha_{\text{post}} + \beta_{\text{post}})^2 (\alpha_{\text{post}} + \beta_{\text{post}} + 1)} = \frac{12}{392} \approx 0.0306, \quad \text{std} \approx 0.175.$$

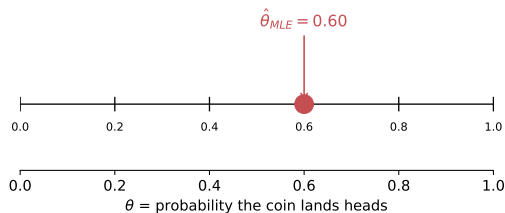
A 95% credible interval (computed from the Beta quantiles) is approximately [0.223, 0.882].

Contrast. The posterior *mode* (0.6) exactly matches the frequentist MLE – this always happens with a uniform (flat) prior. But the Bayesian answer additionally tells us the posterior *mean* (0.5714) and gives a full quantification of how uncertain we should be (std $\approx$ 0.175, a wide 95% credible interval) – information the frequentist point estimate simply does not provide.

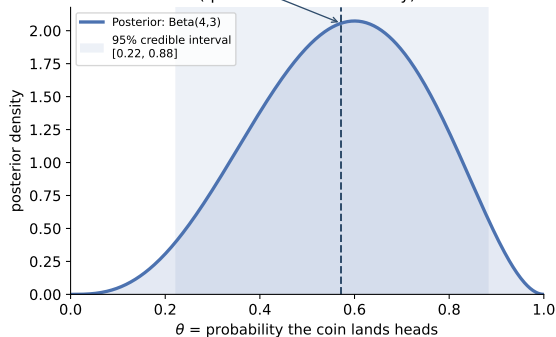
Figure: Frequentist Point Estimate vs. Bayesian Posterior

Toy parameter estimation: 5 coin flips (H,T,H,H,T)

Frequentist: a single point estimate
(no uncertainty attached)



Bayesian: a full distribution over θ
(quantifies the uncertainty)



Left: the frequentist paradigm returns a single point, $\hat{\theta}_{MLE} = 0.6$. Right: the Bayesian paradigm returns a full posterior density, Beta(4,3), together with a mean and a 95% credible interval – the same 5 coin flips, two very different kinds of “answer.”

2.1.3.1 Variational Inference: Turning Sampling into Optimization I

Intuition. When performing Bayesian inference, we want the posterior distribution of the model parameters. **Variational inference (VI)** is an approximate inference technique that obtains an *approximation* to the true posterior, where the approximation comes from a simpler family of distributions (the Gaussian family is a common choice, since it is easy to work with mathematically). Through approximating with a simpler distribution, the sampling problem is transformed into an *optimization* problem: we search for the member of the simple family that is closest to the true posterior.

The picture. We posit a variational family $Q(\mathbf{w}, \nu \mid M)$ over the parameters $\mathbf{w}$ for model M , indexed by variational parameters ν . Starting from initial parameters ν^0 , an optimization procedure moves ν toward the optimal ν^* that makes $Q(\mathbf{w}, \nu^* \mid M)$ as close as possible (in Kullback-Leibler divergence) to the true posterior $\mathbb{P}(\mathbf{w} \mid D, M)$.

2.1.3.1 Variational Inference: Turning Sampling into Optimization II

Kullback-Leibler (KL) Divergence Objective

$$KL(Q(\mathbf{w}, \nu \mid M) \parallel \mathbb{P}(\mathbf{w} \mid D, M)) = \int Q(\mathbf{w}, \nu \mid M) \log \left[\frac{Q(\mathbf{w}, \nu \mid M)}{\mathbb{P}(\mathbf{w} \mid D, M)} \right] d\mathbf{w} \quad (2.2)$$

- $Q(\mathbf{w}, \nu \mid M)$ – the variational (approximating) distribution.
- $\mathbb{P}(\mathbf{w} \mid D, M)$ – the true (target) posterior.
- $KL(\cdot \parallel \cdot) \geq 0$, and $KL = 0$ only when the two distributions are identical.

The catch: Eq. (2.2) still contains the true posterior $\mathbb{P}(\mathbf{w} \mid D, M)$, which itself contains the intractable Bayesian evidence $\mathbb{P}(D \mid M)$ – so the KL divergence is not directly computable either!

2.1.3.1 Variational Inference: Turning Sampling into Optimization III

The Evidence Lower Bound (ELBO)

Expanding Eq. (2.2) using $\mathbb{P}(\mathbf{w} \mid D, M) = \mathbb{P}(\mathbf{w}, D \mid M) / \mathbb{P}(D \mid M)$ and taking expectations with respect to Q gives

$$KL = \underbrace{\mathbb{E}_Q[\log Q(\mathbf{w}, \nu \mid M)] - \mathbb{E}_Q[\log \mathbb{P}(\mathbf{w}, D \mid M)]}_{-\mathbb{L}(D, \nu \mid M) \text{ (tractable)}} + \underbrace{\log \mathbb{P}(D \mid M)}_{\text{intractable}} \quad (2.3)$$

where $\mathbb{L}(D, \nu \mid M)$ is the **evidence lower bound (ELBO)**. Since $KL \geq 0$, Eq. (2.3) shows $\log \mathbb{P}(D \mid M) \geq \mathbb{L}(D, \nu \mid M)$ – the ELBO really is a *lower bound* on the log evidence.

Because the intractable evidence term does not depend on ν , **maximizing the tractable ELBO** $\mathbb{L}(D, \nu \mid M)$ with respect to ν is *equivalent* to minimizing the KL divergence – we have converted an intractable sampling problem into a tractable optimization problem. The optimal ν^* is found by taking the gradient of $\mathbb{L}(D, \nu \mid M)$ with respect to ν and moving in the direction of steepest ascent; the exact optimization formulation depends on the model M and the chosen family Q .

2.1.3.1 Variational Inference: Turning Sampling into Optimization IV

Variational Inference: Pros and Cons

Benefits (vs. MCMC):

- Scales more easily to high dimensions.
- Enables effective mini-batch optimization for large datasets.

Drawback: because we restrict ourselves to one *family* of parametric distributions, bias is introduced by construction – in general we cannot obtain an exact approximation of the true posterior.

2.1.3.2 Markov Chain Monte Carlo: Sampling via a Guided Random Walk I

Using models with difficult-to-compute posteriors, together with approximation procedures to handle them, is the foundation of contemporary Bayesian inference. **MCMC has been the predominant paradigm for approximate inference for more than 50 years:** it was first developed in the 1940s by scientists including Ulam, Von Neumann, Fermi, and Metropolis.

The MCMC recipe. We build a Markov chain on the parameters $\mathbf{w}$, whose stationary distribution *is* the posterior we want. We then take long-run samples from the chain, hoping to gather (approximately independent) samples from that stationary distribution. MCMC's main drawback is its inability to scale well to high dimensions and its comparatively slow execution.

2.1.3.2 Markov Chain Monte Carlo: Sampling via a Guided Random Walk II

Metropolis-Hastings (MH) Acceptance Rule

The seminal, gradient-free Metropolis-Hastings algorithm generates a candidate $\mathbf{w}^*$ from a proposal distribution $T(\mathbf{w}^* | \mathbf{w})$, and accepts or rejects it using the ratio of likelihoods:

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min \left(1, \frac{\pi(\mathbf{w}^*) T(\mathbf{w} | \mathbf{w}^*)}{\pi(\mathbf{w}) T(\mathbf{w}^* | \mathbf{w})} \right) \quad (2.4)$$

- $\pi(\mathbf{w})$ – the stationary target density (here, the posterior) evaluated at $\mathbf{w}$.
- $T(\mathbf{w}^* | \mathbf{w})$ – proposal density for moving from the current state $\mathbf{w}$ to the candidate $\mathbf{w}^*$.

2.1.3.2 Markov Chain Monte Carlo: Sampling via a Guided Random Walk III

Random Walk Metropolis (a common special case)

Using a Gaussian proposal centred at the current state, $\mathcal{N}(\mathbf{w}, \epsilon \Sigma)$, gives the **random walk Metropolis** algorithm (Σ is usually fixed to the identity matrix, leaving only the step size ϵ to tune). Because a Gaussian proposal is symmetric, $T(\mathbf{w} \mid \mathbf{w}^*) = T(\mathbf{w}^* \mid \mathbf{w})$, so Eq. (2.4) simplifies to

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min \left(1, \frac{\pi(\mathbf{w}^*)}{\pi(\mathbf{w})} \right). \quad (2.5)$$

Intuition: always move to a candidate that is *more* probable than the current state; if it is *less* probable, still move there with a probability equal to how much less probable it is (this is what lets the chain fully explore the target distribution rather than getting stuck on a peak).

2.1.3.2 Markov Chain Monte Carlo: Sampling via a Guided Random Walk IV

The drawback of random walk MH. The proposal density in random walk Metropolis-Hastings often induces genuine random-walk behaviour, resulting in elevated **autocorrelations** among the produced samples and slow convergence – large sample sizes are then required because high autocorrelations cause Monte Carlo estimates to converge slowly.

The lineage of more advanced samplers. To lessen this random-walk behaviour, more advanced MCMC methods exploit the structure of the target distribution:

- **First-order** methods use the *gradient* of the target – e.g. **Hamiltonian Monte Carlo (HMC)**, the most widely used MCMC algorithm in the machine learning literature. It suppresses random-walk behaviour by treating parameters as a physical particle moving under simulated Hamiltonian dynamics.
- **Second-order** methods additionally use the *Hessian* (local curvature) – e.g. Riemannian Manifold HMC (RMHMC).
- The **No-U-Turn Sampler (NUTS)** automatically tunes HMC's step size and trajectory length, removing the need for costly manual tuning.

2.1.3.2 Markov Chain Monte Carlo: Sampling via a Guided Random Walk

- Further refinements include **Magnetic HMC (MHMC)** (adds a magnetic field to accelerate convergence and reduce autocorrelation) and the **Separable Shadow Hamiltonian Hybrid Monte Carlo (S2HMC)** family (uses better-conserved “shadow” Hamiltonians to allow larger step sizes).

These advanced samplers will reappear by name in later, application-focused chapters of this book.

Python: A Tiny Metropolis-Hastings Sampler

Goal: recover the $\text{Beta}(4,3)$ posterior from the coin-flip example using a random walk MH sampler (Eq. (2.5)), and check it against the exact conjugate answer.

```
1 import numpy as np
2
3 data = np.array([1, 0, 1, 1, 0])           # 5 coin flips
4
5 def log_target(theta):
6     if theta <= 0.0 or theta >= 1.0:
7         return -np.inf
8     log_prior = 0.0                       # Beta(1,1) = uniform -> constant
9     log_lik = np.sum(data*np.log(theta) + (1-data)*np.log(1-theta))
10    return log_prior + log_lik
11
12 rng = np.random.default_rng(1)
13 n_steps, theta = 20000, 0.5
14 samples = np.empty(n_steps)
15
16 for t in range(n_steps):
17     theta_star = theta + rng.normal(0, 0.15)   # symmetric proposal
18     log_alpha = log_target(theta_star) - log_target(theta)
19     if np.log(rng.uniform()) < log_alpha:      # Eq. (2.5) acceptance
20         theta = theta_star
21     samples[t] = theta
22
23 print("MH posterior mean estimate:", samples[2000:].mean())
24 print("Exact Beta(4,3) posterior mean: 0.5714")
```

2.2 Performance Metrics: Overview I

This section presents the performance metrics used throughout the book to assess the effectiveness of models and algorithms. For both MCMC and variational techniques, we evaluate predictive performance on *unseen* data; for MCMC methods specifically, we additionally evaluate *sampling efficiency*.

Sampling-efficiency metrics (MCMC only)

- **Effective Sample Size (ESS):** the number of *uncorrelated* samples among all the generated samples. The higher the ESS, the more efficient the MCMC technique is at generating quality samples. This book uses the *multivariate* ESS (which accounts for correlations across parameter dimensions) rather than the minimum univariate ESS (which is dominated by whichever single dimension mixes slowest, and ignores relationships between dimensions).
- **ESS normalized by execution time:** accounts for the computational resources needed to generate the samples, penalizing MCMC algorithms that use greater computational resources to produce the same number of uncorrelated samples. The larger this metric, the more efficient the algorithm.

2.2 Performance Metrics: Overview II

Predictive-performance metrics

Classification: accuracy, sensitivity (recall), precision, F-measure, the Receiver Operating Curve (ROC), and the Area Under the ROC (AUC). **Regression:** the coefficient of determination (R^2), Mean Absolute Error (MAE), and Mean Squared Error (MSE).

The following frames define each metric in plain language, with symbols explained and a hand-computed toy example.

Classification Metrics: Confusion Matrix and Rates I

Setup. At a chosen decision threshold, every prediction falls into one of four buckets, forming the **confusion matrix**:

- TP (true positive): predicted positive, actually positive.
- FP (false positive): predicted positive, actually negative.
- TN (true negative): predicted negative, actually negative.
- FN (false negative): predicted negative, actually positive.

Accuracy, Precision, Recall (Sensitivity), F-measure

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F\text{-measure} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Accuracy: overall fraction correct. Precision: of everything flagged positive, how much really was positive. Recall/sensitivity: of everything that really was positive, how much did we catch. F-measure: harmonic mean balancing precision and recall.

Classification Metrics: Confusion Matrix and Rates II

Toy example. 10 examples, true labels y and classifier scores s (predicted probability of the positive class):

$$y = [1, 1, 1, 1, 0, 1, 0, 0, 0, 0], \quad s = [.92, .85, .78, .66, .61, .55, .48, .42, .30, .11].$$

At threshold 0.5: predict positive whenever $s \geq 0.5$, giving predictions $[1, 1, 1, 1, 1, 1, 0, 0, 0, 0]$.

Comparing against y :

$$TP = 5, \quad FP = 1, \quad TN = 4, \quad FN = 0.$$

$$\text{Accuracy} = \frac{5 + 4}{10} = 0.900, \quad \text{Precision} = \frac{5}{5 + 1} = 0.833, \quad \text{Recall} = \frac{5}{5 + 0} = 1.000,$$

$$F\text{-measure} = \frac{2(0.833)(1.000)}{0.833 + 1.000} = 0.909.$$

The ROC Curve and the Area Under the Curve (AUC) I

Receiver Operating Curve (ROC) and AUC

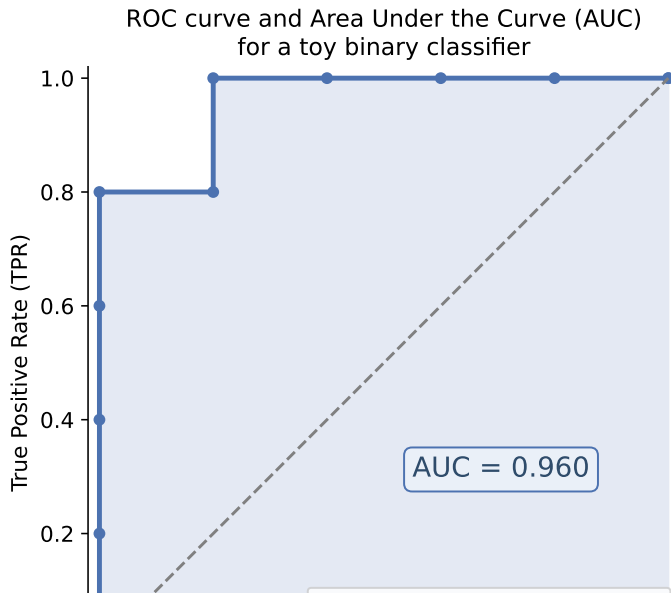
The ROC curve plots the **True Positive Rate** ($TPR = \text{recall}$, y -axis) against the **False Positive Rate** ($FPR = FP/(FP + TN)$, x -axis), as the decision threshold is swept from 1 down to 0. The **Area Under the (ROC) Curve, or AUC**, summarizes the whole curve in a single number between 0 and 1: it indicates the average miss-classification rate across all thresholds. A perfect classifier has $AUC = 1$; a classifier no better than random guessing has $AUC = 0.5$.

Toy example (by hand, trapezoidal rule). Using the same 10 examples as before, sweeping the threshold across every score value and recomputing TPR/FPR at each step traces out a staircase ROC curve. Numerically integrating that staircase with the trapezoidal rule gives

$$AUC \approx 0.960.$$

This is very close to 1: this toy classifier ranks almost all positive examples above almost all negative examples (there is exactly one “inversion”: the negative example scoring 0.61 outranks the positive example scoring 0.55).

The ROC Curve and the Area Under the Curve (AUC) II



Regression Metrics: R^2 , MAE, MSE (and RMSE) I

Mean Squared Error (MSE) and Root MSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

y_i : observed (true) value; $\hat{y}_i$: model's forecast; n : number of observations. MSE (and its square root, RMSE, in the same units as y) quantify the average squared difference between observations and forecasts – *low* values indicate a good model.

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Similar to MSE/RMSE, but penalizes large errors less severely (no squaring). Low MAE indicates a good model.

Regression Metrics: R^2 , MAE, MSE (and RMSE) II

Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

R^2 indicates the percentage of the target variable's variability that the model explains. High R^2 (close to 1) indicates a good model.

Toy example. $y = [3, 5, 2, 7, 4]$, $\hat{y} = [2.5, 5.5, 2, 6, 4.5]$. Residuals: $e = [0.5, -0.5, 0, 1, -0.5]$.

$$\text{MSE} = \frac{0.25 + 0.25 + 0 + 1 + 0.25}{5} = 0.35, \quad \text{RMSE} = \sqrt{0.35} \approx 0.592,$$

$$\text{MAE} = \frac{0.5 + 0.5 + 0 + 1 + 0.5}{5} = 0.5, \quad \bar{y} = 4.2,$$

$$R^2 = 1 - \frac{1.75}{14.8} \approx 0.882.$$

Regression Metrics: R^2 , MAE, MSE (and RMSE) III

Small errors (MSE, RMSE, MAE all modest) and a high R^2 (0.882, i.e. the model explains 88.2% of the variability in y) together indicate a good regression fit.

Python: Computing All Performance Metrics by Hand

```
1 import numpy as np
2
3 # --- classification toy set ---
4 y_true = np.array([1,1,1,1,0,1,0,0,0,0])
5 y_score = np.array([.92,.85,.78,.66,.61,.55,.48,.42,.30,.11])
6 y_pred = (y_score >= 0.5).astype(int)
7
8 TP = np.sum((y_pred==1) & (y_true==1)); FP = np.sum((y_pred==1) & (y_true==0))
9 TN = np.sum((y_pred==0) & (y_true==0)); FN = np.sum((y_pred==0) & (y_true==1))
10
11 accuracy = (TP+TN)/len(y_true)
12 precision = TP/(TP+FP)
13 recall = TP/(TP+FN)
14 f_measure = 2*precision*recall/(precision+recall)
15 print(f"Acc={accuracy:.3f} Prec={precision:.3f} Rec={recall:.3f} F1={f_measure:.3f}")
16
17 # --- regression toy set ---
18 y, y_hat = np.array([3,5,2,7,4]), np.array([2.5,5.5,2,6,4.5])
19 resid = y - y_hat
20 mse, mae = np.mean(resid**2), np.mean(np.abs(resid))
21 r2 = 1 - np.sum(resid**2)/np.sum((y-y.mean())**2)
22 print(f"MSE={mse:.4f} RMSE={np.sqrt(mse):.4f} MAE={mae:.4f} R2={r2:.4f}")
```

Output: Acc=0.900 Prec=0.833 Rec=1.000 F1=0.909 and MSE=0.3500 RMSE=0.5916
MAE=0.5000 R2=0.8818 – matching every hand computation on the previous slides exactly.

2.3 Conclusion I

This chapter introduced:

- the **Bayesian framework** and how it differs from the frequentist paradigm (Table 2.1);
- typical **Bayesian machine learning models** – Bayesian logistic regression, Bayesian mixture models, Gaussian processes, Bayesian neural networks, and Bayesian formulations of economic models – plus two further tools (normalizing flows, and deeper BNN machinery) previewed for later chapters;
- how to **train** these models, focusing on the two main approaches: **variational inference** (an optimization problem, via the ELBO) and **Markov Chain Monte Carlo** (a sampling problem, via a guided random walk); and
- the **performance metrics** used throughout the book: multivariate ESS and ESS/time for MCMC sampling efficiency; accuracy, precision, recall, F-measure, ROC and AUC for classification; and R^2 , MAE, MSE for regression.

2.3 Conclusion II

Looking ahead. In the next chapter, we calibrate the Stochastic Alpha Beta Rho (SABR) stochastic volatility model using Bayesian methods, and show the benefits of this in terms of uncertainty quantification – without loss of model accuracy. Every term introduced in this chapter (posterior, likelihood, prior, evidence, MCMC, variational inference, Gaussian processes, normalizing flows, Bayesian neural networks, and the performance metrics above) will be used without further re-explanation from here on.

Recap: Key Vocabulary Checklist I

- **Bayesian machine learning:** fitting a model *and* obtaining a full distribution over its uncertainty, not just one best-fit answer.
- **Prior / likelihood / posterior:** belief before data / probability of data given parameters / belief after data.
- **Bayesian evidence:** the normalizing constant $\mathbb{P}(D \mid M)$; useful for model selection.
- **Variational inference:** approximate the posterior with a simpler distribution Q , and find the best member of that family by maximizing the ELBO (an optimization problem).
- **MCMC:** draw samples from the posterior by running a cleverly designed random walk whose stationary distribution is the posterior (a sampling problem); Metropolis-Hastings is the seminal algorithm, HMC/NUTS/MHMC/S2HMC are its modern descendants.
- **Gaussian process:** a non-parametric “distribution over functions.”
- **Normalizing flow:** build a complex distribution by pushing a simple one (e.g. Gaussian) through invertible transformations.
- **Bayesian neural network:** a neural network with a distribution over its weights, not fixed numbers.
- **Performance metrics:** (multivariate) ESS and ESS/time for MCMC efficiency; accuracy / precision / recall / F-measure / ROC / AUC for classification; R^2 / MAE / MSE for regression.

References

-  W. T. Mongwe, R. Mbuva, and T. Marwala, *Bayesian Machine Learning in Quantitative Finance: Theory and Practical Applications*. Springer, 2025.
-  W. K. Hastings, "Monte Carlo Sampling Methods Using Markov Chains and Their Applications," *Biometrika*, 57(1), 97–109, 1970.
-  M. D. Hoffman and A. Gelman, "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo," *JMLR*, 15(1), 1593–1623, 2014.
-  D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, "Variational Inference: A Review for Statisticians," *JASA*, 112(518), 859–877, 2017.
-  T. Marwala, R. Mbuva, and W. T. Mongwe, *Hamiltonian Monte Carlo Methods in Machine Learning*. Elsevier, 2023.
-  D. Vats, J. M. Flegal, and G. L. Jones, "Multivariate Output Analysis for Markov Chain Monte Carlo," *Biometrika*, 106(2), 321–337, 2019.
-  J. P. Bharadiya, "A Review of Bayesian Machine Learning Principles, Methods, and Applications," *International Journal of Innovative Science and Research Technology*, 8(5), 2033–2038, 2023.
-  D. J. C. MacKay, "A Practical Bayesian Framework for Backpropagation Networks," *Neural Computation*, 4(3), 448–472, 1992.